

Spark RDD Assessment — MCQs & Scenario-Based Questions

Section A: Multiple Choice Questions (MCQs)

1. What does RDD stand for?

- a) Resilient Distributed Dataset
- b) Reliable Data Distribution
- c) Random Data Duplication
- d) Resilient Data Definition

2. RDDs are:

- a) Mutable and can be changed in place
- b) Immutable - transformations create new RDDs
- c) Only usable with SQL
- d) Automatically persisted in memory by default

3. Which of the following is a *transformation*, not an *action*?

- a) collect()
- b) count()
- c) map()
- d) reduce()

4. Spark transformations are:

- a) Eagerly evaluated immediately when called
- b) Lazily evaluated - executed only when an action is triggered
- c) Executed only on the driver node
- d) Not part of the DAG

5. Which operation is a *wide* transformation (causes a shuffle)?

- a) map()
- b) filter()
- c) groupByKey()
- d) union()

6. Which of these creates an RDD from an existing Scala/Python collection?

- a) sc.textFile()
- b) sc.parallelize()
- c) sc.wholeTextFiles()
- d) sc.hadoopFile()

7. Which persistence level stores data in memory only, and recomputes from lineage if it doesn't fit?

- a) MEMORY_AND_DISK
- b) DISK_ONLY
- c) MEMORY_ONLY
- d) OFF_HEAP

8. What is the default number of partitions when reading a file with sc.textFile() (assuming no second argument given)?

- a) Always exactly 1

- b) Determined by the number of executors only
- c) Based on HDFS block size / input splits (or defaultMinPartitions)
- d) Always equal to number of cores on the driver

11. Which action returns the entire RDD content to the driver program as a local collection?

- a) take(n)
- b) collect()
- c) saveAsTextFile()
- d) foreach()

12. What happens if you call collect() on a very large RDD that doesn't fit in driver memory?

- a) Spark automatically writes it to disk
- b) It throws an OutOfMemoryError on the driver
- c) It silently truncates the data
- d) Nothing , Spark handles it transparently

13. persist() / cache() on an RDD is useful when:

- a) The RDD is used only once in the program
- b) The RDD is reused multiple times across actions, to avoid recomputation
- c) You want to make the RDD mutable
- d) You want to force a shuffle

16. What does flatMap() do differently from map()?

- a) flatMap() cannot take a lambda/function
- b) flatMap() flattens the output , each input can map to zero, one, or many output elements
- c) flatMap() only works on numeric RDDs
- d) There is no difference

17. Which of these is NOT a valid way to create an RDD?

- a) From a collection using parallelize()
 - b) From an external file using textFile()
 - c) By transforming an existing RDD
 - d) By directly instantiating new RDD() with data in PySpark
-

Section B: Worked Examples (map & filter only)

Example 1 — map(): double every number

Output: [2, 4, 6, 8, 10]

```
numbers = sc.parallelize([1,2,3,4,5])
```

```
double = numbers.map(lambda x:x*2)
```

```
double.collect()
```

Example 2 — filter(): keep only even numbers

Output: [2, 4, 6]

```
numbers = sc.parallelize([1,2,3,4,5,6])
evens = numbers.filter(lambda x:x%2==0)
evens.collect()
```

Example 3 — chaining map() and filter()

```
numbers = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8])
# Output: [18, 21, 24] (6*3=18, 7*3=21, 8*3=24)
numbers678 = numbers.filter(lambda x:x>5)
table3 = numbers678.map(lambda x:x*3)
table3.collect()
—
numbers678_table3=numbers.filter(lambda x:x>5).map(lambda x:x*3)
numbers678_table3.collect()
```

Example 4 — map() on strings

```
# Output: ['ALICE', 'BOB', 'CHARLIE']
names = sc.parallelize(["alice","bob","charlie"])
upper_names = names.map(lambda x:x.upper())
upper_names.collect()
```

Example 5 — filter() on strings

Find all the words with length greater or equal than 5 from a list of words.

```
# Output: ['apple', 'banana', 'watermelon']
words= sc.parallelize(["apple","two","banana","car","home","watermelon"])
len5_words = words.filter(lambda x:len(x)>=5)
len5_words.collect()
```

Section C: Scenario-Based Coding Questions

For each scenario, write the PySpark (or Scala) code. Assume sc is the SparkContext and, where relevant, name your variables clearly. **Use only map(), filter(), flatMap(), and basic actions (collect(), count(), sum(), first(), take(), max(), min()) — no reduceByKey, groupByKey, or aggregateByKey needed for these.**

Scenario 1 — Splitting Lines into Words You have a text file sample.txt with multiple lines of text. Write RDD code to:

a) Read the file into an RDD of lines

```
sample = sc.textFile("/sample.txt")
```

b) Split each line into individual words and produce a single flat RDD of all words (hint: flatMap)

```
flated_sample = sample.flatMap(lambda x:x.split(" "))
```

c) Convert every word to lowercase

```
lower_flated_sample = flated_sample.map(lambda x:x.lower())
```

d) Count how many words are longer than 4 characters

```
len4_lower_flated_sample = lower_flated_sample.filter(lambda x:len(x)>4)
```

```
len4_lower_flated_sample .count()
```

Scenario 2 — Filtering and Counting You have an RDD of integers: numbers = sc.parallelize([4, 7, 2, 9, 15, 22, 3, 18, 11]). Write code to:

a) Return only the even numbers

```
even_numbers = numbers.filter(lambda x:x%2==0)
```

b) Count how many numbers are greater than 10

```
greater_10_numbers = numbers.filter(lambda x:x>10)
```

```
greater_10_numbers.count()
```

c) Find the sum of all numbers

```
numbers.sum()
```

Scenario 3 — Sales Data Filtering You have an RDD of tuples representing sales: (region, amount), e.g. sales = sc.parallelize([("East",100), ("West",200), ("East",150), ("North",50), ("West",300)]) Write code to:

a) Extract just the amount values into a new RDD using map()

```
sales_values = sales.map(lambda x: x[1])
```

b) Filter to keep only sales where amount > 100

```
sales_values_greater_100 = sales_values .filter(lambda x:x>100)
```

c) Find the total of all filtered amounts and the single highest amount overall (Note: computing totals *per region* would normally use reduceByKey — we'll cover that later. For now, just work with the amounts as a whole.)

```
total_sales_greater_100 = sales_values_greater_100.sum()
```

```
max_sales_value = sales_values.max()
```

Scenario 4 — Joining Two Datasets

You have two RDDs:

```
employees = sc.parallelize([(101,"Alice"), (102,"Bob"), (103,"Charlie")])
```

```
salaries = sc.parallelize([(101,50000), (102,60000), (104,45000)])
```

Write code to: a) Perform an inner join on employee ID and display (id, name, salary)

```
complete_employees = employees.join(salaries)
```

```
cleaned_complete_employees = complete_employees.map(lambda x: (x[0],x[1][0],x[1][1]))
```

b) Perform a left outer join from employees and explain what happens to employee 103 and 104 in each case.

```
employees.leftOuterJoin(salaries)
```

It gives all the employees (101, 102 and 103) with their salaries, in case that an employee doesn't join with a salary its salary would be None, 103 case. The id 104 doesn't appear, because this id isn't in employees.

Scenario 5 — Lazy Evaluation Trace Given the following code, list the sequence of actions/transformations and state **at which exact line Spark actually starts computing anything**, and why.

```
rdd1 = sc.parallelize([1,2,3,4,5])
```

```
rdd2 = rdd1.map(lambda x: x * 2)
```

```
rdd3 = rdd2.filter(lambda x: x > 4)
```

```
print("Defined transformations")
```

```
result = rdd3.collect()
```

```
print(result)
```

Sequence of actions:

```
print("Defined transformations") -> print(result) -> reading what is result: result = rdd3.collect() ->
rdd3 = rdd2.filter(lambda x: x > 4) -> rdd2 = rdd1.map(lambda x: x * 2) -> rdd1 =
sc.parallelize([1,2,3,4,5]) -> after finished what it is necessary for compute result it start doing the
compute rdd1 = sc.parallelize([1,2,3,4,5]) -> rdd2 = rdd1.map(lambda x: x * 2) -> rdd3 =
rdd2.filter(lambda x: x > 4) -> result = rdd3.collect() -> print(result)
```

First Spark is going to print "Defined transformations" because it doesn't need any compute. With the print(result) it needs to compute everything to get the value of result, looking for how to compose it.

Scenario 6 — map() vs filter(): Picking the Right Transformation You have temps = sc.parallelize([21, 35, 40, 15, 28, 33, 8]) representing daily temperatures (°C).

a) Write code using `map()` to convert every value to Fahrenheit ($F = C * 9/5 + 32$)

```
f_temps = temps.map(lambda x:x* 9/5 + 32)
```

b) Write code using `filter()` to keep only days where the Celsius temperature was above 30

```
temps_above_30 = temps.filter(lambda x:x>30)
```

c) In your own words, explain the difference between what `map()` returns vs what `filter()` returns (size of output RDD, type of output).

`map` is going to transform each value of the RDD and `filter` is going to keep certain values of the RDD that match a condition

Scenario 7— Pass/Fail Filtering Given scores = `sc.parallelize([("Alice",90), ("Bob",42), ("Charlie",70), ("David",35), ("Eve",100)])` (name, score out of 100, pass mark = 40):

a) Write code using `filter()` to return only the students who passed

```
passed = scores.filter(lambda x:x[1]>=40)
```

b) Write code using `map()` to convert the list into (name, "Pass") or (name, "Fail") pairs

```
trans_scores = scores.map(lambda x: (x[0], "Pass" if x[1] >= 40 else "Fail" ))
```

c) Count how many students failed (*Grouping/averaging by subject would use `aggregateByKey` — that's for a later session.*)

```
failed = scores.filter(lambda x:x[1]<40)
```

```
failed.count()
```
